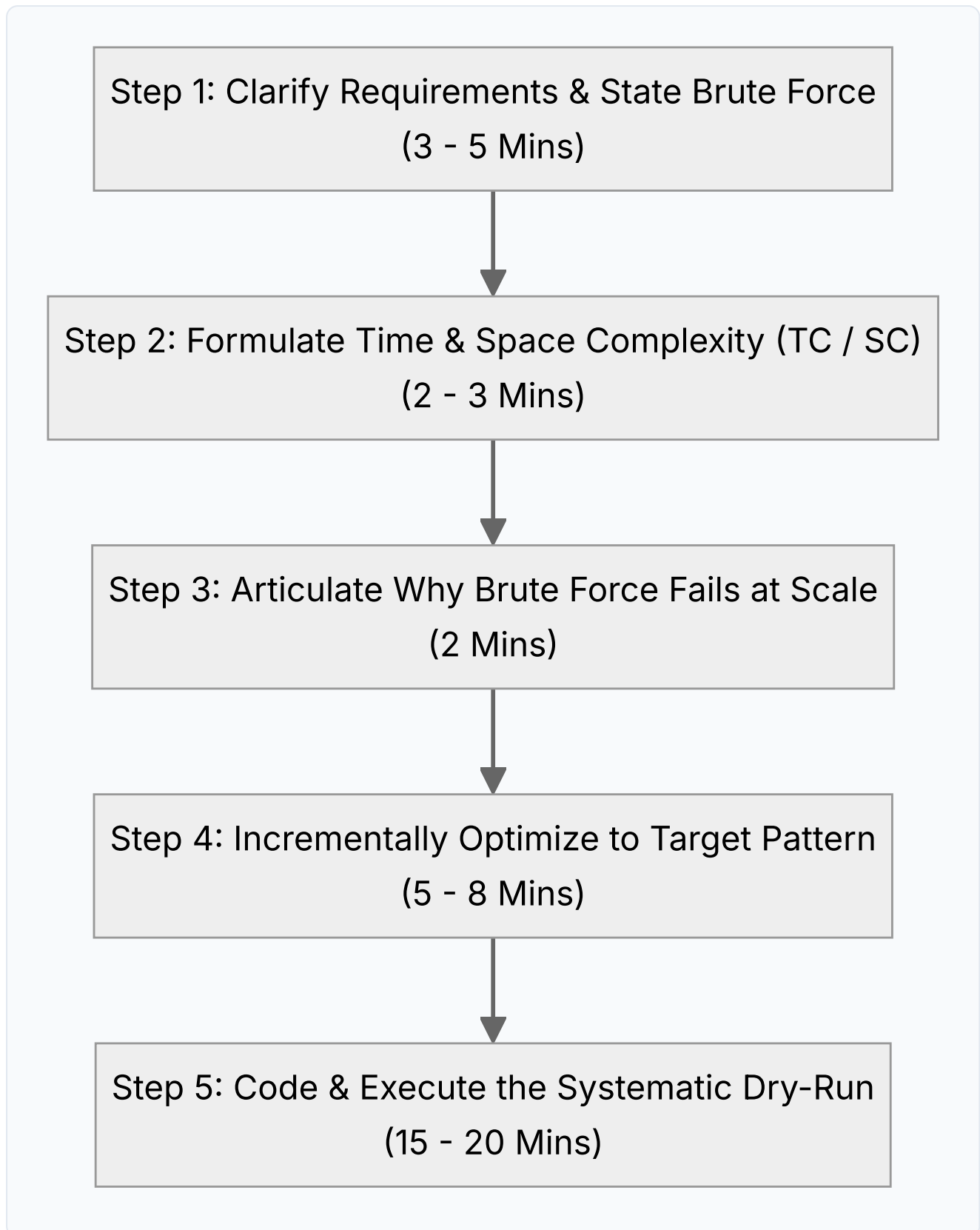


DSA Interview Playbook: Communication, Optimization & The Dry-Run Framework

Core Objective: Master the behavioral and technical execution framework required to clear Data Structures & Algorithms rounds at top-tier Product-Based Companies (Google, Meta, Uber, Amazon).

1. The 5-Step Interview Communication Loop

Interviews at top companies are **collaborative problem-solving simulations**, not solitary coding tests. The interviewer is evaluating your thought process, communication clarity, and ability to handle edge cases.



Step 1: Clarify & Propose the Brute Force

- **Never jump directly into coding.** Ask about input constraints ($N \leq 10^3$ vs $N \leq 10^6$), memory limits, whether inputs are sorted, if duplicates exist, and what should be returned for empty inputs.

- Immediately articulate the simplest brute-force algorithm:

"To ensure we have a solid baseline, the brute-force approach would be to iterate through all possible subarrays using a nested loop, computing the sum for each. This would guarantee correctness."

Step 2: Discuss TC & SC of Brute Force

- State the time and space complexity explicitly using Big-O notation:

"This brute-force approach requires examining $\frac{N(N+1)}{2}$ subarrays, resulting in $O(N^2)$ time complexity and $O(1)$ auxiliary space complexity."

Step 3: Convey Clearly Why Brute Force Breaks Under High Traffic

- Interviewers love when you connect algorithmic complexity to real-world infrastructure constraints:

"If our input size N is 10^5 , an $O(N^2)$ algorithm performs roughly 10^{10} operations. Standard CPU architectures execute approximately 10^8 operations per second, meaning this query would take ~100 seconds to respond, timing out our HTTP gateway and dropping user requests."

Step 4: Incremental Optimization

- Identify the bottleneck: *Are we repeating duplicate calculations? (Memoization/DP). Are we doing linear lookups? (Hash Map/Set). Is the data monotonic? (Two Pointers/Binary Search).*
- Propose the optimal strategy and get interviewer agreement **before typing code**.

Step 5: The Structured Dry-Run (Most Important Step)

- **Never say "I'm done" the second you type the last line of code.**
- Walk through an explicit trace table with sample inputs, tracking variable values step-by-step.

2. The Universal Dry-Run Trace Matrix

When presenting code to the interviewer, draw out a trace matrix on the whiteboard or scratchpad:

Example Problem: Two Sum (Find indices summing to target = 9 in `nums = [2, 7, 11, 15]` **)**



3. High-Frequency Pattern Reference Cheat Sheet

Pattern	Common Triggers / Keywords	Typical Data Structure	Time / Space
Sliding Window	Contiguous subarrays/substrings, "longest/shortest window with $\text{sum} \leq K$ "	Two pointers (<code>left</code> , <code>right</code>), Hash Map	$O(N) / O(K)$
Two Pointers	Sorted array, searching pairs, palindrome validation, partitioning in-place	Pointers at <code>left = 0</code> , <code>right = N-1</code>	$O(N) / O(1)$
Fast & Slow Pointers	Cycle detection in linked list, middle of linked list, circular arrays	<code>slow = slow.next</code> , <code>fast = fast.next.next</code>	$O(N) / O(1)$
Monotonic Stack	"Next Greater Element", "Largest Rectangle in Histogram", stock span	Stack holding indices with strictly monotonic values	$O(N) / O(N)$
Top K Elements	Find the K most frequent items, K -th largest element in stream	Min-Heap or Max-Heap (<code>heapq</code>) of size K	$O(N \log K) / O(K)$
Intervals Overlap	Meeting rooms, calendar conflicts, merge overlapping ranges	Sort intervals by <code>start_time</code>	$O(N \log N) / O(N)$
Binary Search on Answer	"Minimize the maximum capacity", monotonic condition function	<code>low</code> , <code>high</code> , <code>mid</code> , verify <code>is_valid(mid)</code>	$O(\log(\text{range}) \cdot N)$
Topological Sort	Course scheduling, task dependency DAG, build order	In-degree array + BFS Queue (Kahn's Algorithm)	$O(V + E) / O(V + E)$
Trie (Prefix Tree)	Autocomplete, word search, prefix matching	Multi-way tree (<code>children = dict</code> , <code>is_end = bool</code>)	$O(L)$ query ($L = \text{length}$)
0/1 Knapsack / DP	Subset sum, partitioning, optimal selection with weight constraints	2D / 1D DP table (bottom-up iteration)	$O(N \cdot W)$

Generated Dry-Run Traces

A dry run is only useful if it is faithful to the code. The repository includes a tracer that **generates** the trace from the actual solution, so it cannot drift from what the code does:

How to read it: each row is a line about to execute, and the values next to it are the variables that changed since the previous line (so they show the effect of the line above). Here you can watch `seen` fill with `{2: 0}`, then `complement` become `2` on the second pass and the function return `[0, 1]`.

Use it three ways: trace the reference solution on the example to learn the mechanism, trace **your own stub** with `--stub` to see where it diverges, and trace a failing fuzz input (the harness prints the counterexample) to find the exact line where your variable goes wrong. Arguments after the problem number are Python literals, for example `python practice/trace.py 20 "[1,3,5,7,9]" 7`.

Further Reading

- [Big-O cheat sheet](#)
- [Python time complexity](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. What do you say first when given a coding problem?

Restate it, ask clarifying questions about inputs and constraints, and confirm examples and edge cases.

▼ 2. Why describe brute force before optimising?

It proves correctness of your understanding and the bottleneck suggests the optimisation.

▼ 3. How do you dry-run your solution?

Trace a small example line by line, track variable values, then test edge cases.

▼ 4. What do you do when stuck?

Think aloud, simplify the problem, try a small case, and ask for a hint rather than going silent.